SEO PACKAGE

1. SEO Title

SQL LIMIT and OFFSET Explained: Pagination, the Deep-Page Slowdown, and Keyset Pagination

2. SEO Subtitle

Learn how to page through millions of rows correctly with `LIMIT` and `OFFSET`, why deep offsets crawl, and when to switch to keyset (seek) pagination.

3. Featured Quote

> "`OFFSET 1000000` isn't skipping a million rows for free — the database walks every one of them just to throw them away."

4. Meta Description

Master SQL LIMIT and OFFSET for pagination, understand why deep OFFSET is slow, and learn keyset pagination as the scalable alternative. (154 characters)

5. URL Slug

`sql-limit-offset-pagination-keyset-explained`

6. Keywords

SQL LIMIT, SQL OFFSET, SQL pagination, keyset pagination, seek pagination, cursor pagination, deep offset performance, ORDER BY pagination, page size formula, database pagination, LIMIT vs OFFSET, OFFSET FETCH, PostgreSQL pagination, MySQL pagination, SQL Server pagination, paginate large tables, off-by-one pagination bug, index scan pagination, backend pagination, SQL performance tuning

7. Tags

`SQL`, `Databases`, `Pagination`, `Backend Development`, `PostgreSQL`, `MySQL`, `SQL Server`, `Performance`, `Software Engineering`, `Coding Interview`, `Data Structures`, `Query Optimization`, `Learn SQL`, `Web Development`

---

8. Introduction

Every application that lists things eventually hits the same wall. You have a table with a million rows, a screen that shows ten, and a user who wants to scroll. You cannot dump a million records into a browser and hope for the best — you have to hand them out a page at a time.

That is what `LIMIT` and `OFFSET` are for. They are two of the smallest clauses in SQL, but they quietly power almost every paginated list you have ever used: search results, feeds, order histories, admin dashboards. Learn them well and you can slice any result set into clean, predictable pages.

Interviewers love this topic because it separates people who *memorized* syntax from people who *understand* what the database is actually doing. Anyone can type `LIMIT 10`. The real question is the follow-up: *"Your pagination works fine on page 2 but times out on page 50,000. Why, and how do you fix it?"* If you can answer that, you have demonstrated that you think about performance, indexes, and trade-offs — exactly the skills that matter in real backend work.

By the end of this article you will know how to page through large datasets correctly, why deep pagination gets slow, and when `OFFSET` is the wrong tool entirely.

---

9. Problem Overview

Here is the problem stated plainly.

A `SELECT` query hands you back **rows** — each row is one record, like one customer or one order. A single table can hold millions of them, but you almost never want the whole set at once. You want a *slice*: "give me ten results, starting from the right place."

To carve out that slice you need two independent controls:

1. **A ceiling on how many rows come back.** No matter how many rows match, return at most *N*. This is `LIMIT`.
2. **A starting point deeper into the result set.** Skip past the rows you have already shown before you start collecting. This is `OFFSET`.

Combine the two and you get **pagination**: skip some rows, then take a fixed number. That is the whole game.

But there are three subtleties that trip up almost everyone:

- Without an explicit sort order, the "same" page can return different rows on every refresh.
- `OFFSET N` means *skip N rows*, not *start at row N* — a classic off-by-one trap.
- The deeper the page, the slower the query, because the database still has to walk past every skipped row.

We will handle all three.

---

10. Example

Imagine a `books` table:

id	title	published
1	The Pragmatic Programmer	1999
2	Clean Code	2008
3	Designing Data-Intensive Applications	2017
4	The Mythical Man-Month	1975
5	Refactoring	1999
6	Code Complete	2004
7	Working Effectively with Legacy Code	2004

`LIMIT` on its own — deal me the top three:

```
SELECT id, title
FROM books
ORDER BY id
LIMIT 3;
```

Returns rows `1, 2, 3`. The database keeps the first three and ignores the rest. It does not even look at the dropped rows — it just stops early once it has what you asked for.

`OFFSET` with `LIMIT` — skip three, then take three:

```
SELECT id, title
FROM books
ORDER BY id
LIMIT 3 OFFSET 3;
```

Returns rows `4, 5, 6`. The `OFFSET 3` steps over the first three rows, then `LIMIT 3` collects the next three starting at row `4`.

Notice the pattern: `OFFSET 3` did **not** start at row 3. It *skipped* three rows and started at the fourth. This skip-then-take combination is the heart of pagination.

---

11. Intuition

Think of a deck of cards.

`LIMIT` is you saying, *"Deal me the top three cards, then stop."* You do not care about the other forty-nine. The dealer stops the moment your hand is full. That is a ceiling on how many rows come back.

`OFFSET` is the opposite move. Picture a queue of people. `OFFSET 3` is you saying, *"Skip the first three people in line — I'll start with the fourth."* You are moving your starting point further down the list before you begin.

Put them together and pagination falls out naturally. A page shows ten items, so `LIMIT` is `10`. To reach page three, you have to walk past the first two pages — that is twenty rows — so `OFFSET` is `20`. The formula practically writes itself:

```
OFFSET = page_size × (page_number − 1)
```

For page 3 with a page size of 10: `10 × (3 − 1) = 20`. Clean.

The experienced-engineer move is to notice *what the database actually has to do* to honor an `OFFSET`. `LIMIT` is cheap — the engine stops early. But `OFFSET` is not free. To skip a hundred thousand rows, the engine still has to **produce** a hundred thousand rows in sorted order and then discard them. Once you internalize that asymmetry, the deep-page slowdown stops being a mystery and the alternative solution becomes obvious.

---

12. Brute Force Solution

Idea: Use `LIMIT` and `OFFSET` directly with the page formula. This is the standard, textbook approach and it is genuinely the right choice for most tables.

Algorithm:

1. Fix your page size (say, 10).

2. Compute `OFFSET = page_size × (page_number − 1)`.
3. Always add an `ORDER BY` on a stable, unique column so pages don't shuffle.
4. Run `SELECT ... ORDER BY id LIMIT page_size OFFSET offset`.

```
-- Page 3, 10 rows per page
SELECT id, title, published
FROM books
ORDER BY id
LIMIT 10 OFFSET 20;
```

Advantages:

- Dead simple to write and reason about.
- Lets you **jump to any page** — page 1, page 47, page 900 — with a single formula.
- Works the same way whether the user goes forward, backward, or teleports to page 50.

Disadvantages:

- **It drags on deep pages.** Reaching `OFFSET 100000` means the database walks through a hundred thousand rows just to throw them away.
- Rows can **shift between pages** if data is inserted or deleted while a user is paging (an item slides from page 2 onto page 1, and someone sees it twice or misses it).

Complexity:

- **Time:** `O(offset + limit)` — the cost grows linearly with how deep the page is. Page 1 is instant; page 10,000 is not.
- **Space:** `O(limit)` for the returned rows.

---

13. Optimal Solution

For deep pagination, the fix is **keyset pagination** — also called **seek** or **cursor** pagination.

The insight: instead of counting *how many rows to skip*, remember *where you left off*. Keep the sort key of the last row you saw, and ask the database for rows **after** that value. Because the sort column is indexed, the engine jumps straight to that position in the index and reads forward — no wasted scanning of rows you'll discard.

Here is the difference visually.

Offset pagination — the deeper you go, the more you waste:

Page	What the DB does	Work
1	Read rows 1–10	10 rows touched
100	Produce rows 1–1000, discard 990, return 10	1,000 rows touched
10,000	Produce rows 1–100,000, discard 99,990, return 10	100,000 rows touched

Keyset pagination — flat cost on every page:

Page	What the DB does	Work
Any	Seek index to `id > last_seen_id`, read next 10	10 rows touched

The mechanics: your first query grabs the first page and you record the last `id` you saw. The next request passes that `id` back, and the query becomes "give me rows *after* this one."

```
-- First page
SELECT id, title
FROM books
ORDER BY id
LIMIT 10;

-- Next page: last id from the previous page was 10
SELECT id, title
FROM books
WHERE id > 10          -- seek straight to the right spot via the index
ORDER BY id
LIMIT 10;
```

The database uses the index on `id` to jump directly to the first row greater than `10` and reads ten rows forward. It never touches the rows you already saw. That is why keyset stays fast on page 1 and page 1,000,000 alike.

The trade-off: you can only move **next** and **previous**, because all you know is "the last row I saw." You cannot teleport to "page 4,782" — there is no offset to compute. For infinite-scroll feeds, that is exactly the interaction you want anyway.

> **Tie-breaker note:** if your sort column isn't unique (say you sort by `created_at`), two rows can share a value and the seek becomes ambiguous. Fix it by sorting on the column **plus** a unique tiebreaker like the primary key, and comparing the pair: `WHERE (created_at, id) > (:last_created_at, :last_id)`.

---

14. Python Code

Pagination usually lives half in SQL and half in your application layer, so here is a clean, PEP 8-compliant Python helper that builds both styles. It uses parameterized queries — never format user

input into SQL strings.

```
"""Pagination query builders for offset and keyset (seek) styles."""

from dataclasses import dataclass


def offset_page_sql(page_number: int, page_size: int = 10) -> tuple[str, dict]:
    """Build an OFFSET-based page query. Good for small/admin tables.

    Lets you jump to any page, but slows down on deep pages.
    """
    if page_number < 1:
        raise ValueError("page_number is 1-based and must be >= 1")

    offset = page_size * (page_number - 1)  # the page formula
    sql = (
        "SELECT id, title, published "
        "FROM books "
        "ORDER BY id "          # ORDER BY is mandatory for stable pages
        "LIMIT %(limit)s OFFSET %(offset)s"
    )
    return sql, {"limit": page_size, "offset": offset}


@dataclass
class Cursor:
    """The 'where I left off' marker for keyset pagination."""
    last_id: int | None = None  # None means "start from the beginning"


def keyset_page_sql(cursor: Cursor, page_size: int = 10) -> tuple[str, dict]:
    """Build a keyset (seek) page query. Stays fast on any page.

    Only supports next-page navigation, not arbitrary jumps.
    """
    params = {"limit": page_size}
    where = ""
    if cursor.last_id is not None:
        where = "WHERE id > %(last_id)s "  # seek past what we've seen
        params["last_id"] = cursor.last_id

    sql = (
        "SELECT id, title, published "
        "FROM books "
        f"{where}"
        "ORDER BY id "
        "LIMIT %(limit)s"
    )
    return sql, params
```

```
if __name__ == "__main__":
    # Self-check: the page formula and cursor logic behave as expected.
    _, p3 = offset_page_sql(page_number=3, page_size=10)
    assert p3["offset"] == 20, "page 3 @ size 10 must skip 20 rows"

    _, p1 = offset_page_sql(page_number=1, page_size=10)
    assert p1["offset"] == 0, "page 1 must skip nothing"

    first_sql, first_params = keyset_page_sql(Cursor(last_id=None))
    assert "WHERE" not in first_sql, "first keyset page has no cursor filter"

    next_sql, next_params = keyset_page_sql(Cursor(last_id=10))
    assert "id > %(last_id)s" in next_sql, "later pages seek past last_id"
    assert next_params["last_id"] == 10

    try:
        offset_page_sql(page_number=0)
    except ValueError:
        pass
    else:
        raise AssertionError("page 0 should be rejected")

    print("All pagination checks passed.")
```

---

15. Code Walkthrough

- **`offset_page_sql`** turns a human page number into a database offset. The line `offset = page_size * (page_number - 1)` is the pagination formula from earlier — page 1 skips nothing, page 3 skips 20.
- The guard `if page_number < 1` rejects nonsense input up front. Pages are 1-based; a page 0 or negative page is a caller bug, and failing loudly beats returning a silently wrong slice.
- The SQL string always includes `ORDER BY id`. This is not optional — it is the single most important line for correctness, which we'll see why in Edge Cases.
- **`Cursor`** is a tiny dataclass holding `last_id`. A `None` value means "we haven't started yet," so the first page has no filter.
- **`keyset_page_sql`** conditionally adds `WHERE id > %(last_id)s`. On the first page there is no cursor, so the `WHERE` clause is empty and you simply get the first N rows. On every later page, that clause seeks past everything already seen.
- Both functions return `(sql, params)` and use **placeholders** (`%(name)s`), never string interpolation of values. This is how you avoid SQL injection — the driver binds parameters safely.
- The `__main__` block is a runnable self-check: it verifies the offset math, confirms the first keyset page has no filter, confirms later pages seek correctly, and confirms bad input is rejected.

16. Dry Run

Let's page through the `books` table with a page size of 3, using keyset pagination.

Request 1 — first page. `Cursor(last_id=None)`.

The builder produces:

```
SELECT id, title, published FROM books ORDER BY id LIMIT 3;
```

The database returns rows `1, 2, 3`. Your application records the last id seen: `3`.

Request 2 — next page. `Cursor(last_id=3)`.

The builder produces:

```
SELECT id, title, published FROM books WHERE id > 3 ORDER BY id LIMIT 3;
```

The engine uses the index on `id` to jump straight to the first row greater than `3`, then reads three rows: `4, 5, 6`. It never re-reads rows `1–3`. New last id: `6`.

Request 3 — next page. `Cursor(last_id=6)`.

```
SELECT id, title, published FROM books WHERE id > 6 ORDER BY id LIMIT 3;
```

Only row `7` remains, so you get a single row back. Fewer rows than the limit is your signal that you've reached the end.

Notice that at no point did the database's work grow. Whether this was page 1 or page 1,000, each request touched at most three rows — the flat cost that makes keyset scale.

---

17. Complexity Analysis

Offset pagination:

- **Time:** `O(offset + limit)`. To return your `limit` rows, the engine must first generate and discard `offset` rows in sorted order. On page 1 the offset is 0 and it's instant; on page 10,000 the offset is 100,000 and the cost grows right along with it. The slowness is *inherent to the*

operation, not a missing index — even a perfectly indexed sort still has to walk past the skipped rows.
- **Space:** `O(limit)` — only the returned page is materialized for the client.

Keyset pagination:

- **Time:** `O(log n + limit)` when the sort column is indexed. The `log n` is the index seek to find your starting position (a B-tree lookup); the `limit` is reading the page forward. Crucially, this is **independent of how deep the page is** — page 1 and page 1,000,000 cost the same.
- **Space:** `O(limit)` for the page, plus a tiny constant for the cursor value you carry between requests.

The takeaway: offset cost scales with *depth*, keyset cost is *flat*. That single difference is the entire reason keyset exists.

---

18. Alternative Solutions

Beyond the two main approaches, a couple of variations show up in real systems:

- **Cached total count + offset.** If you need to show "Page 3 of 4,912," you need a total row count, which is itself expensive on large tables. A common trick is to cache an approximate count (Postgres exposes estimates via `pg_class.reltuples`) rather than running `COUNT(*)` on every page load.
- **Deferred join (offset optimization).** You can soften offset's pain by paginating over just the indexed key first, then joining back for the full row: `SELECT * FROM books JOIN (SELECT id FROM books ORDER BY id LIMIT 10 OFFSET 100000) AS page USING (id);`. The engine skips rows in a narrow index instead of hauling full rows, which is cheaper — but the cost still grows with depth. It's a mitigation, not a cure.

How they compare:

Approach	Jump to any page?	Deep-page speed	Best for
`LIMIT` / `OFFSET`	✅ Yes	❌ Slows down	Small datasets, admin tables
Keyset (seek)	❌ Next/prev only	✅ Always fast	Huge feeds, infinite scroll
Deferred join	✅ Yes	⚠️ Better, still degrades	Deep offset you can't avoid

Rule of thumb: **small dataset or admin table → use `OFFSET`. Huge feed that scrolls forever → use keyset.**

---

19. Edge Cases

- **No `ORDER BY`.** This is the big one. Without a sort, the database is free to return rows in *any* order it likes, and that order can change between queries. The same "page 2" shows different rows on each refresh — users see duplicates or miss records. Always paginate with an explicit `ORDER BY` on a stable column.
- **Non-unique sort column.** Sorting by `created_at` alone breaks keyset when two rows share a timestamp. Add a unique tiebreaker: `ORDER BY created_at, id`.
- **Offset past the end.** `OFFSET 1000` on a 50-row table returns zero rows — not an error, just an empty page. Your UI should handle "no results" gracefully.
- **Empty table.** Both approaches return an empty result set cleanly. No special handling needed.
- **Data changing mid-page.** With offset pagination, an insert or delete between page loads shifts every subsequent row, causing skipped or duplicated records. Keyset is more resilient because it anchors to a value, not a position.
- **Very deep offset (`OFFSET 1000000`).** Technically valid, but this is where offset pagination crawls. If you're routinely hitting offsets this deep, that's the signal to switch to keyset.

---

20. Common Mistakes

1. **Paginating without `ORDER BY`.** The number-one bug. Pages appear stable in testing (small data, consistent order) and then shuffle in production. If you use `LIMIT` and `OFFSET`, an `ORDER BY` on a unique column is mandatory.
2. **Reading `OFFSET N` as "start at row N."** `OFFSET 1` does not start at row one — it *skips* one row and starts at row two. Say it out loud: offset is *how many to skip*, not *where to begin*.
3. **Getting the page formula backward.** It's `page_size × (page_number − 1)`, not `page_size × page_number`. Forgetting the `− 1` skips an entire first page — the classic off-by-one that shows the wrong page to every user.
4. **Sorting on a non-unique column without a tiebreaker.** Ties make row order ambiguous, which quietly corrupts both offset and keyset paging. Always append a unique column to the sort.
5. **Assuming every database uses the same syntax.** Postgres and MySQL use `LIMIT ... OFFSET ...`. SQL Server and the SQL standard use `OFFSET ... ROWS FETCH NEXT ... ROWS ONLY`. Same idea, different words — check your dialect.
6. **Using deep offset on a hot, huge table.** It works in the demo and times out under load. If pages go deep, reach for keyset before it becomes a 3am incident.

---

21. Interview Questions

- Why must you always pair `LIMIT` / `OFFSET` with `ORDER BY`?
- Your list endpoint is fast on page 2 but times out on page 50,000. What's happening and how do you fix it?
- Explain keyset pagination. What does it give up compared to offset pagination?
- How would you implement "next page" and "previous page" with keyset pagination?
- How do you paginate correctly when the sort column has duplicate values?
- How would you show a total page count (`Page 3 of 4,912`) without running an expensive `COUNT(*)` on every request?
- What's the difference between `LIMIT ... OFFSET` and `OFFSET ... FETCH`, and which databases use which?
- How does keyset pagination behave when rows are inserted or deleted while a user is paging?

---

22. Similar Problems

- **[Design an API rate limiter / paginated API]** — pagination is a core piece of any list endpoint; the cursor pattern here is exactly how "cursor-based pagination" works in GraphQL and REST APIs.
- **Top K elements** — `ORDER BY ... LIMIT K` is SQL's version of the classic "find the top K" pattern, and the same intuition about not sorting more than you need applies.
- **Merge K sorted lists** — keyset pagination across multiple partitioned tables is essentially a K-way merge over sorted streams.
- **Binary search on sorted data** — the index seek that makes keyset fast (`WHERE id > last_seen`) is a B-tree lookup, the database's cousin of binary search over a sorted structure.

These are related because they all lean on the same core idea: when data is *ordered*, you can reach the part you want without scanning everything before it.

---

23. Key Takeaways

- `LIMIT` caps how many rows come back; `OFFSET` skips rows before you start collecting.
- Together they give you pagination: `OFFSET = page_size × (page_number − 1)`.
- **Always** add `ORDER BY` on a unique column, or your pages will shuffle.
- `OFFSET N` means *skip N rows*, not *start at row N*.

- Deep offset is slow because the database walks and discards every skipped row — cost grows with page depth.
- For huge, deep-scrolling datasets, use **keyset (seek) pagination**: remember the last row and ask for rows after it. It stays fast on any page but only does next/previous.
- Dialects differ: `LIMIT` / `OFFSET` (Postgres, MySQL) vs `OFFSET` / `FETCH` (SQL Server, standard SQL).

---

24. Watch the Video

Want to see `LIMIT`, `OFFSET`, and keyset pagination in action — with the row-by-row animations that make the "skip-then-take" idea click instantly? Watch the full lesson here: {LINK}

Seeing the arrow step over skipped rows and land on the right page makes the whole concept stick in a way that reading alone can't.

---

25. About the Series

This is **Lesson 5** of the SQL course in the **Fun with Learning Technology** series — a growing set of short, practical lessons that take backend and database concepts from "I've heard of it" to "I can use it confidently in an interview and in production." Each lesson builds real intuition first, then shows the code, then covers the trade-offs and edge cases the way an experienced engineer actually thinks about them. No fluff, no memorization — just the *why* behind each idea.

---

26. Call To Action

If this made pagination click for you:

- 👍 **Like the video on YouTube** — that's what tells the algorithm to show it to more developers.
- 🔔 **Subscribe** so you don't miss the next SQL lesson.
- 📩 **Subscribe to the Substack** for the written deep-dives that go beyond each video.
- 💬 **Comment** with the topic you want covered next — or your worst pagination bug story.
- 🔁 **Share** it with a teammate who's still using `OFFSET 1000000` and wondering why it's slow.

SOCIAL MEDIA

LinkedIn Post

Most developers learn `LIMIT` and `OFFSET` in about five minutes, then get burned by them in production six months later. Here's what the five-minute version leaves out.

`LIMIT` caps how many rows a query returns. `OFFSET` skips rows before it starts collecting. Together they build pagination, using a simple formula:

```
OFFSET = page_size × (page_number − 1)
```

That's the easy part. The parts that actually matter in real systems:

1️⃣ **Always pair them with `ORDER BY`.** Without an explicit sort, the database can return rows in any order it likes — so "page 2" shows different rows on every refresh. This bug hides perfectly in testing and only appears under real traffic.

2️⃣ **`OFFSET N` means skip N rows, not start at row N.** A small distinction that causes very real off-by-one bugs.

3️⃣ **Deep offset doesn't scale.** To reach `OFFSET 100000`, the database walks through 100,000 rows just to throw them away. Page 1 is instant; page 10,000 crawls. The cost grows with page depth — it's inherent to the operation, not a missing index.

The fix for deep pagination is **keyset (seek) pagination**: instead of counting rows to skip, you remember the last row you saw and ask for rows after it. The database seeks straight there via the index, so it stays fast on any page. The trade-off is you only get next/previous, not arbitrary page jumps.

Rule of thumb: small or admin table → `OFFSET`. Huge feed that scrolls forever → keyset.

Understanding *why* a query is slow is what separates engineers who write SQL from engineers who own it.

What's the deepest offset you've seen in a production query? 👇

SQL #Databases #Backend #SoftwareEngineering #Performance

Twitter/X Thread

1/ `LIMIT` and `OFFSET` are two of the smallest words in SQL — and two of the most misunderstood.

Here's how to paginate a million rows correctly, and why `OFFSET 1000000` crawls. 🧵

2/ Start simple. `LIMIT 3` = "deal me the top 3 rows, then stop."

The database returns at most 3 rows and ignores the rest. It doesn't even look at the dropped rows — it just stops early.

`LIMIT` is a ceiling on how many rows come back.

3/ `OFFSET` is the opposite move. `OFFSET 3` = "skip the first 3 rows, start at the 4th."

So `LIMIT 3 OFFSET 3` skips 3 rows, then grabs the next 3.

Skip-then-take. That's the heart of pagination.

4/ Page size 10, want page 3? Skip the first 2 pages = 20 rows.

The formula: `OFFSET = page_size × (page_number − 1)`

Page 3 → 10 × (3 − 1) = 20. Clean.

5/ Trap #1: reading `OFFSET 1` as "start at row 1."

It doesn't. `OFFSET 1` SKIPS one row and starts at row 2.

Offset = how many to skip. Not where to begin. Say it out loud — it saves real bugs.

6/ Trap #2: paginating without `ORDER BY`.

Without a sort, the DB returns rows in ANY order it likes — and it can change between queries. Same "page 2" shows different rows on every refresh.

Always add `ORDER BY` on a unique column.

7/ Now the big one. `OFFSET` is overrated for deep pages.

To reach `OFFSET 100000`, the DB walks through 100,000 rows just to throw them away.

Page 1 = instant. Page 10,000 = crawl. Cost grows with depth.

8/ The fix: keyset (seek) pagination.

Instead of skipping rows, remember the id of the last row you saw and ask for rows AFTER it:

`WHERE id > 10 ORDER BY id LIMIT 10`

The DB seeks straight there via the index. No wasted scanning.

9/ Keyset stays fast on page 1 AND page 1,000,000.

The trade-off: you only get next/previous. No jumping to "page 4,782" — there's no offset to compute.

For infinite-scroll feeds, that's exactly the interaction you want.

10/ Rule of thumb:

• Small dataset or admin table → `OFFSET` • Huge feed that scrolls forever → keyset

And a heads-up: Postgres/MySQL use `LIMIT` / `OFFSET`; SQL Server & standard SQL use `OFFSET` / `FETCH`. Same idea, different words.

That's pagination done right. ✅

Facebook Post

Ever wonder how apps show you "page 3" of search results without loading all million records at once? 🤔

That's `LIMIT` and `OFFSET` — two tiny SQL clauses doing a huge amount of work. `LIMIT` says "give me at most 10 rows," `OFFSET` says "skip the first 20," and together they slice a giant table into neat little pages.

But there's a catch most people learn the hard way: the deeper you page, the slower it gets — because to reach page 10,000, the database has to walk past every single row before it just to throw them away. 😅

In the new lesson I break down exactly why that happens, and show the faster alternative (keyset pagination) that stays quick no matter how deep you scroll. Beginner-friendly, with visuals that make it click.

Come learn it with me 👇

Reddit Summary

LIMIT/OFFSET pagination: the parts that actually bite you in production

Sharing a breakdown of SQL pagination that goes past the syntax, since deep pagination is one of those things that works fine in dev and quietly falls over under load.

The basics: `LIMIT` caps returned rows, `OFFSET` skips rows first. Page formula is `OFFSET = page_size × (page_number − 1)`.

Three things worth knowing:

- **Always use `ORDER BY`.** Without a defined sort, the engine can return rows in any order, and it can differ between runs. Pages shuffle on refresh — a bug that hides well in testing.
- **`OFFSET N` skips N rows**, it doesn't start *at* row N. Small distinction, real off-by-one bugs.
- **Deep offset doesn't scale.** `OFFSET 100000` still forces the DB to produce and discard 100,000 rows in sorted order. Cost grows with depth — it's inherent to the operation, not just a missing index.

The alternative for deep/infinite-scroll cases is **keyset (seek) pagination**: keep the sort key of the last row and query `WHERE id > last_seen ORDER BY id LIMIT n`. With an index on the sort column it's a B-tree seek, so cost is flat regardless of page depth (`O(log n + limit)` vs offset's `O(offset + limit)`). Downside: next/prev only, no arbitrary page jumps. If your sort column isn't unique, add a tiebreaker and compare tuples: `WHERE (created_at, id) > (:last_ts, :last_id)`.

Rule of thumb: small/admin tables → offset (simple, jump anywhere). Huge feeds → keyset. Also note dialect differences: `LIMIT` / `OFFSET` (Postgres, MySQL) vs `OFFSET` / `FETCH` (SQL Server, ANSI SQL).

Happy to answer questions on the deferred-join trick for cases where you're stuck with deep offset.

GITHUB README

```
# SQL Pagination: LIMIT, OFFSET & Keyset (Seek) Pagination

Lesson 5 of the SQL course — how to page through large datasets correctly,
why deep `OFFSET` gets slow, and when to switch to keyset pagination.
```

```
## Problem

A `SELECT` can return millions of rows, but a screen shows ten. You need to
hand out results one page at a time — a *slice* of the result set, starting at
the right place. That's pagination.

## Intuition

- **`LIMIT`** — a ceiling on how many rows come back. Like "deal me the top 3
  cards, then stop." The database stops early and never looks at the rest.
- **`OFFSET`** — a starting point further down the list. Like "skip the first 3
  people in line, start at the 4th." It's *how many rows to skip*, not the row
  number to start at.
- **Together** — skip, then take. That's pagination.

Page formula:
```

OFFSET = page_size × (page_number − 1)

```
The catch: `OFFSET` isn't free. To skip N rows, the database still produces N
rows in sorted order and discards them. The deeper the page, the slower it gets.

## Approach

### Offset pagination (simple, jump to any page)

Best for small datasets and admin tables. Always pair with `ORDER BY` on a
unique column, or pages shuffle between refreshes.

```sql
-- Page 3, 10 rows per page
SELECT id, title, published
FROM books
ORDER BY id
LIMIT 10 OFFSET 20;
```

Keyset / seek pagination (fast on any page)

Best for huge, infinite-scroll feeds. Remember the last row seen and ask for rows after it — the index seeks straight there. Only supports next/previous.

```
-- Next page after last seen id = 10
SELECT id, title, published
FROM books
WHERE id > 10
```
```

```
ORDER BY id
LIMIT 10;
```

If the sort column isn't unique, add a tiebreaker:

```
WHERE (created_at, id) > (:last_created_at, :last_id)
ORDER BY created_at, id
LIMIT 10;
```

Python Solution

```
from dataclasses import dataclass


def offset_page_sql(page_number: int, page_size: int = 10) -> tuple[str, dict]:
    """OFFSET-based page. Jump anywhere, but slows on deep pages."""
    if page_number < 1:
        raise ValueError("page_number is 1-based and must be >= 1")
    offset = page_size * (page_number - 1)
    sql = (
        "SELECT id, title, published FROM books "
        "ORDER BY id LIMIT %(limit)s OFFSET %(offset)s"
    )
    return sql, {"limit": page_size, "offset": offset}


@dataclass
class Cursor:
    last_id: int | None = None


def keyset_page_sql(cursor: Cursor, page_size: int = 10) -> tuple[str, dict]:
    """Keyset (seek) page. Fast on any page; next-only."""
    params = {"limit": page_size}
    where = ""
    if cursor.last_id is not None:
        where = "WHERE id > %(last_id)s "
        params["last_id"] = cursor.last_id
    sql = (
        f"SELECT id, title, published FROM books {where}"
        "ORDER BY id LIMIT %(limit)s"
    )
    return sql, params
```

Complexity

Approach	Time	Jump to any page?	Best for
`LIMIT` / `OFFSET`	`O(offset + limit)`	✅ Yes	Small / admin tables
Keyset (seek)	`O(log n + limit)`	❌ Next/prev only	Huge scrolling feeds

Space is `O(limit)` for both. Offset cost scales with page depth; keyset cost is flat.

Gotchas

- Always paginate with `ORDER BY` on a unique column.
- `OFFSET N` skips N rows — it does not start *at* row N.
- Dialects differ: `LIMIT` / `OFFSET` (Postgres, MySQL) vs `OFFSET` / `FETCH` (SQL Server, standard SQL).

Video

📺 Watch the full lesson: {LINK}

Article

📖 Full written deep-dive with diagrams, dry run, edge cases, and interview questions in the accompanying article.

Part of the **Fun with Learning Technology** series. ```

Quick Review

When do you reach for LIMIT and OFFSET?

When you need pagination, top-N results, or a bounded slice of a large result set. See LIMIT n OFFSET m → think 'skip m rows, take n'.

Time cost of LIMIT n OFFSET m?

Roughly O(m + n): the database still scans and discards the first m rows before returning n. Large offsets get progressively slower.

Space complexity of LIMIT/OFFSET?

O(n) for the returned rows (plus sort buffers if ORDER BY isn't index-backed). OFFSET itself adds no extra storage — it just skips.

Most common LIMIT/OFFSET bug?

Using LIMIT/OFFSET without ORDER BY. Row order is undefined, so pages overlap or drop rows. Always pair with a stable ORDER BY.

OFFSET pagination vs keyset pagination?

OFFSET scans and throws away skipped rows (slow deep in). Keyset filters WHERE id > last_seen ORDER BY id — index-seekable and fast at any depth.

Page 3 with page size 10 — what OFFSET?

OFFSET = (page − 1) × size = 20, LIMIT 10. Off-by-one trap: pages are 1-indexed but offset is 0-indexed.

---

sql Lesson 5: Limiting Results — LIMIT and OFFSET (Limiting results involves restricting the number of rows returned by a query, while offsetting allows skipping a specific number of records. These techniques are essential for managing large datasets by processing or displaying data in manageable portions. You reach for these clauses whenever you implement pagination, performance optimization, or need to retrieve a subset like 'top N' records.) — free Python cheat sheet